



Labs - Fondation du Machine learning

Version :	1.0
Date de version :	10/08/2026
Date de revue :	10/08/2026
Créé par :	Data Engineer
Appartenant à :	Head of AI
Approuvé par :	Head of AI
Normes et règlements applicables :	ISO/IEC 27001 - ISO 9001 - ISO/IEC 42001 - RGPD
Niveau de confidentialité :	TLP Orange - Restreint

Historique des modifications

Date	Version	Créé par	Description du changement
10/08/26	1.0	Data Engineer	Première version

Historique des revues

Date	Version	Revu par	Description de la revue
10/08/26	1.0	Data Engineer	Première version

Prédiction de Prix Immobiliers - Régression Linéaire

Bienvenue dans ce Lab pratique dédié aux fondations du Machine Learning ! Ce travail est conçu pour vous guider pas à pas dans l'univers de la modélisation prédictive. En incarnant le rôle d'un Data Analyst au sein d'une agence immobilière, vous allez apprendre à transformer des données brutes en insights hautement stratégiques. Préparez-vous à explorer, nettoyer, modéliser et valider vos théories scientifiques avec rigueur !

Partie I : Régression Linéaire Simple

La régression linéaire simple est le point de départ de la modélisation statistique. L'objectif est d'établir une relation mathématique directe entre une unique variable explicative et une variable cible continue.

Description des Données : “house-prices.csv”

Pour cette première partie, vous travaillerez avec un extrait simplifié du marché immobilier comprenant deux colonnes :

- **area** : La superficie d'une maison (mesurée en mètres carrés ou pieds carrés). Elle sert de variable explicative (ou prédicteur).
- **price** : Le prix de vente de la maison. C'est votre variable cible (target), que vous cherchez à modéliser et prédire.

Rappels théoriques de la Régression Simple

L'équation théorique du modèle s'écrit :

$$Price = c + m \times Area + \xi$$

Où :

- c : l'ordonnée à l'origine,
- m : la pente (le coefficient de régression), représentant la variation moyenne du prix pour chaque unité de superficie supplémentaire.
- ξ : le terme d'erreur (ou résidu), qui capte la part de variabilité non expliquée par le modèle.

Travaux Pratiques

Étape 1 : Exploration et Visualisation des Données

- ☐ Charger votre fichier de données de régression simple à l'aide de la bibliothèque Pandas.

```
df = pd.read_csv("house-prices.csv")
print("Données chargées depuis le fichier CSV!")
```

✓ 0.0s Python

Données chargées depuis le fichier CSV!

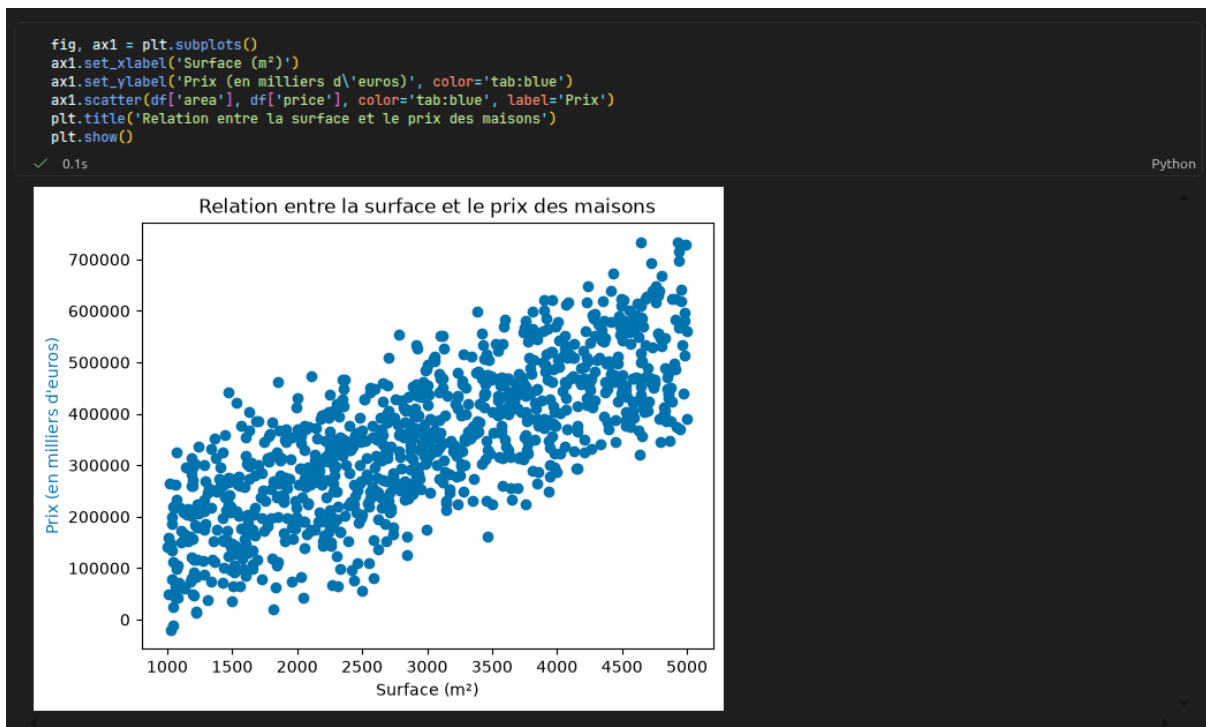
- ☐ Afficher les premières lignes du jeu de données (.head()) pour inspecter la structure des données.

```
df.head()
```

✓ 0.0s Python

	area	price
0	1567	177067.50
1	3345	511748.20
2	3722	404096.67
3	2783	357916.31
4	2949	362409.54

- ☐ Générer un graphique de dispersion (scatter plot) avec Matplotlib ou Seaborn pour visualiser la relation entre area (axe X) et price (axe Y). Confirmer visuellement si l'hypothèse d'une tendance linéaire semble valide.



Étape 2 : Séparation des Données et Modélisation

- ☐ Diviser le jeu de données en un ensemble d'entraînement (Train set) et un ensemble de test (Test set) avec un ratio de 80/20 afin de prévenir tout risque de surapprentissage.

```
from sklearn.model_selection import train_test_split
```

✓ 0.9s Python

```
X = df['area'].values
y = df['price'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Shape totale des données : {df.shape}")
print(f"Nombre d'échantillons d'entraînement : {len(X_train)}")
print(f"Nombre d'échantillons de test : {len(X_test)}")
```

✓ 0.0s Python

```
Shape totale des données : (1000, 2)
Nombre d'échantillons d'entraînement : 800
Nombre d'échantillons de test : 200
```

- ☐ Instancier un objet de régression linéaire de scikit-learn (LinearRegression).

```
from sklearn.linear_model import LinearRegression
```

✓ 0.0s Python

```
model = LinearRegression()
✓ 0.0s
```

Python

- ☐ Ajuster le modèle (`.fit()`) sur le jeu d'entraînement pour calculer les coefficients optimaux par la méthode des MCO.

```
model.fit(X_train, y_train)
✓ 0.0s
```

Python

Étape 3 : Évaluation et Interprétation des Résultats

- ☐ Calculer le coefficient de détermination (R^2) sur l'ensemble d'entraînement et l'ensemble de test.

```
from sklearn.metrics import r2_score
✓ 0.0s
```

Python

```
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
print(f"Coefficient de détermination R² : {r2:.4f}")
✓ 0.0s
```

Python

Coefficient de détermination R² : 0.5945

- ☐ Extraire et afficher l'intercept (c) et le coefficient associé à la variable area (m).

```
print("m (pente) :", model.coef_[0])
print("b (ordonnée à l'origine) :", model.intercept_)
✓ 0.0s
```

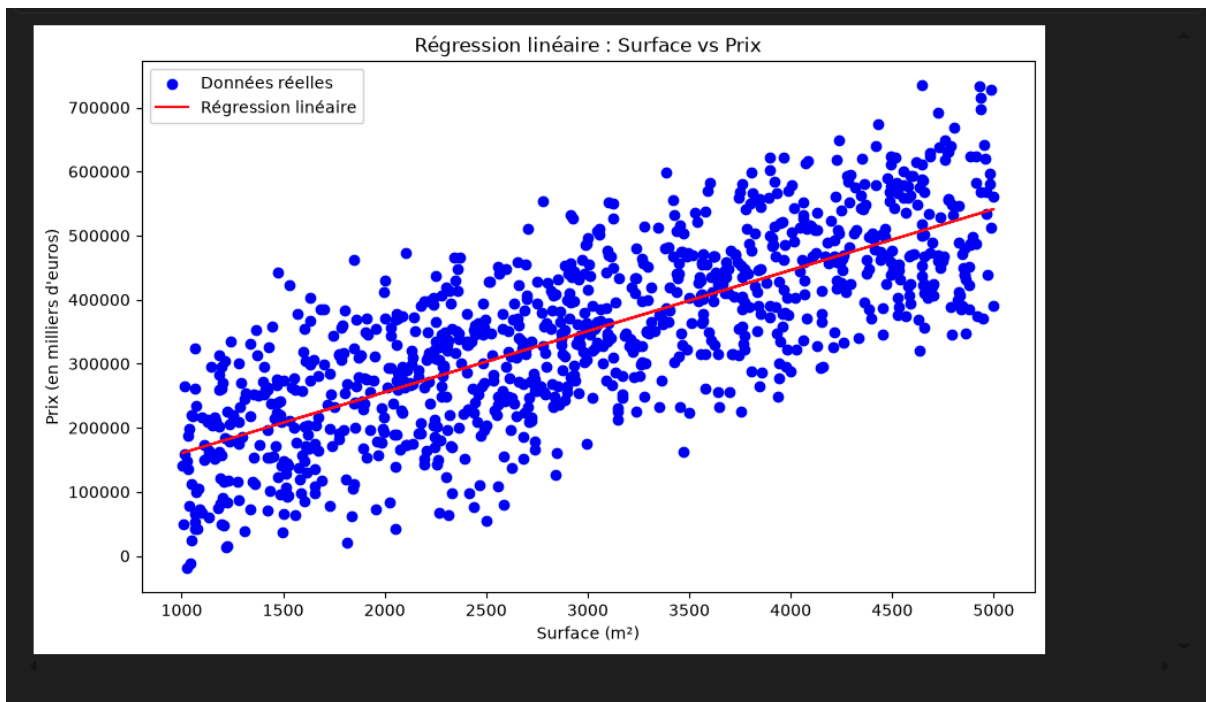
Python

m (pente) : 95.2280295860481
b (ordonnée à l'origine) : 65145.494366672705

- ☐ Interpréter concrètement le sens physique de m : si la superficie augmente d'une unité, de combien le prix augmente-t-il en moyenne ?
- ☐ Tracer la droite de régression linéaire sur le graphique de dispersion des données de test pour évaluer l'alignement du modèle avec la réalité.

```
plt.figure(figsize=(10, 6))
plt.scatter(df['area'], df['price'], color='blue', label='Données réelles')
plt.plot(df['area'], predictions, color='red', label='Régression linéaire')
plt.xlabel('Surface (m²)')
plt.ylabel('Prix (en milliers d\'euros)')
plt.title('Régression linéaire : Surface vs Prix')
plt.legend()
plt.show()
✓ 0.3s
```

Python



Partie II : Régression Linéaire Multiple

Dans la réalité, le prix d'une propriété dépend d'une multitude de facteurs. La régression linéaire multiple étend le concept simple en intégrant plusieurs prédicteurs simultanément pour capter la complexité du marché.

Description des Données : “house_prices_v2.csv”

Ce jeu de données complet contient des informations enrichies sur diverses propriétés immobilières :

- **Square_Footage** : La superficie habitable de la maison en pieds carrés.
- **Num_Bedrooms** : Le nombre de chambres dans la maison.
- **Num_Bathrooms** : Le nombre de salles de bain.
- **Year_Built** : L'année de construction de la maison (indiquant l'usure ou la modernité).
- **Lot_Size** : La taille totale du terrain mesurée en acres.

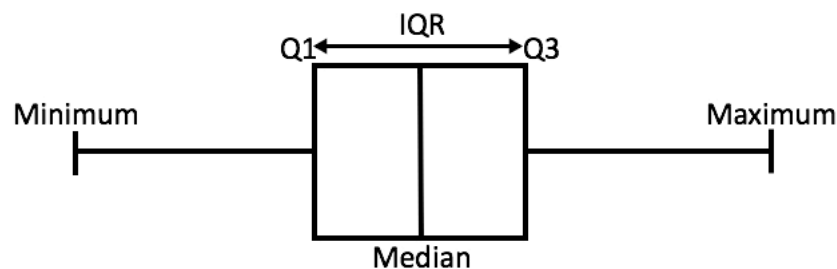
- **Garage_Size** : La capacité du garage (nombre de voitures acceptées).
- **Neighborhood_Quality** : Une évaluation de la qualité du quartier sur une échelle de 1 à 10.
- **House_Price** (Variable Cible) : Le prix final de la maison que vous cherchez à prédire.

Rappels théoriques de la Régression Simple

- Détection des Valeurs Atypiques (Outliers) par l'IQR

Avant d'entraîner un modèle multivarié, il est indispensable de traiter les valeurs aberrantes qui peuvent fortement influencer et fausser l'estimation par les MCO. L'Écart Interquartile (IQR) est calculé ainsi :

- Q_1 (25ème percentile) : Médiane de la moitié inférieure des données.
- Q_3 (75ème percentile) : Médiane de la moitié supérieure des données.
- $IQR = Q_3 - Q_1$



- $Min = Q_1 - 1.5 \times IQR$
- $Max = Q_3 + 1.5 \times IQR$

Outlier si:

$$val > Max \text{ ou } val < Min$$

- Théorème de Gauss-Markov et Validation des Résidus

Pour que les coefficients estimés par les MCO soient reconnus comme les

meilleurs estimateurs linéaires non biaisés (BLUE), les résidus du modèle ($\xi_i = y_i - \hat{y}_i$) doivent valider les hypothèses du théorème de Gauss-Markov :

- **Linéarité** : La relation entre la variable cible et les variables explicatives est linéaire.
- **Indépendance des erreurs** : Les erreurs d'observations distinctes ne sont pas corrélées.
- **Espérance nulle** : La moyenne théorique des erreurs est nulle ($E[\xi] = 0$).
- **Homoscédasticité** : La variance des erreurs est constante pour toutes les valeurs des prédicteurs. Une variance inégale est appelée hétéroscédasticité.
- **Absence de colinéarité parfaite** : Les variables explicatives ne doivent pas être fortement corrélées entre elles (ce qui rendrait l'estimation instable).
- **Normalité des erreurs** : Pour valider les tests de significativité, les erreurs doivent suivre une loi normale $N(0, \sigma^2)$.

Travaux Pratiques

Étape 1 : Nettoyage et Préparation des Données

- ☐ Calculer l'IQR pour identifier et écarter les outliers.

```
print("--- ÉTAPE 1 : Nettoyage des données ---")

# Calcul de l'IQR sur la variable cible pour exclure les valeurs aberrantes
Q1 = df['House_Price'].quantile(0.25)
Q3 = df['House_Price'].quantile(0.75)
IQR = Q3 - Q1

Min = Q1 - 1.5 * IQR
Max = Q3 + 1.5 * IQR

# Filtrage des outliers
df_clean = df[(df['House_Price'] >= Min) & (df['House_Price'] <= Max)].copy()
print(f"Données initiales : {df.shape[0]} lignes | Après filtrage IQR : {df_clean.shape[0]} lignes\n")

✓ 0.0s Python
```

--- ÉTAPE 1 : Nettoyage des données ---
Données initiales : 1000 lignes | Après filtrage IQR : 1000 lignes

- ☐ Traiter les valeurs manquantes résiduelles en appliquant une imputation robuste (médiane de la colonne concernée) pour préserver la cohérence statistique des données.

```
df.isna().sum()
0.0s
Python
```

Square_Footage	0
Num_Bedrooms	0
Num_Bathrooms	0
Year_Built	0
Lot_Size	0
Garage_Size	0
Neighborhood_Quality	0
House_Price	0
dtype: int64	

Étape 2 : Feature Engineering et Prévention du Data Leakage

- ☐ Découper de manière stricte le jeu de données en ensembles d'entraînement et de test (`train_test_split`) avant toute étape de transformation ou de calcul de statistiques globales pour éliminer tout risque de data leakage.
- ☐ Standardiser vos variables explicatives quantitatives à l'aide de `StandardScaler`. Veiller à appliquer le `.fit_transform()` exclusivement sur l'ensemble d'entraînement, et le `.transform()` sur l'ensemble de test.
- ☐ (Optionnel) Encapsuler l'ensemble de vos transformations (imputation, standardisation, régression) dans un Pipeline scikit-learn pour garantir la robustesse et la reproductibilité de votre flux de travail.

```
print("--- ÉTAPE 2 : Prévention du Leakage & Prétraitement ---")

# Séparation des features (X) et de la cible (y)
X = df_clean.drop('House_Price', axis=1)
y = df_clean['House_Price']

# Découpage STRICT en Train/Test AVANT l'imputation et la standardisation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Création du Pipeline scikit-learn (Imputation par la médiane + Standardisation)
preprocessor = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Application stricte : .fit_transform() sur Train, .transform() sur Test
X_train_scaled = preprocessor.fit_transform(X_train)
X_test_scaled = preprocessor.transform(X_test)

# Reconversion en DataFrame pour conserver le nom des variables (indispensable pour l'interprétation)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns, index=X_test.index)

0.0s
Python
```

--- ÉTAPE 2 : Prévention du Leakage & Prétraitement ---

Étape 3 : Modélisation et Diagnostics de Gauss-Markov

```
print("--- ÉTAPE 3 : Modélisation et Diagnostics ---")

# Matrice de corrélation (Calculée sur le Train set)
plt.figure(figsize=(8, 6))
sns.heatmap(X_train.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1, fmt=".2f")
plt.title("Matrice de Corrélation (Vérification de la Colinéarité)")
plt.show()

# Ajout de la constante (ordonnée à l'origine) requise par statsmodels
X_train_sm = sm.add_constant(X_train_scaled)
X_test_sm = sm.add_constant(X_test_scaled)

# Entraînement du modèle OLS (Ordinary Least Squares)
modele_sm = sm.OLS(y_train, X_train_sm).fit()

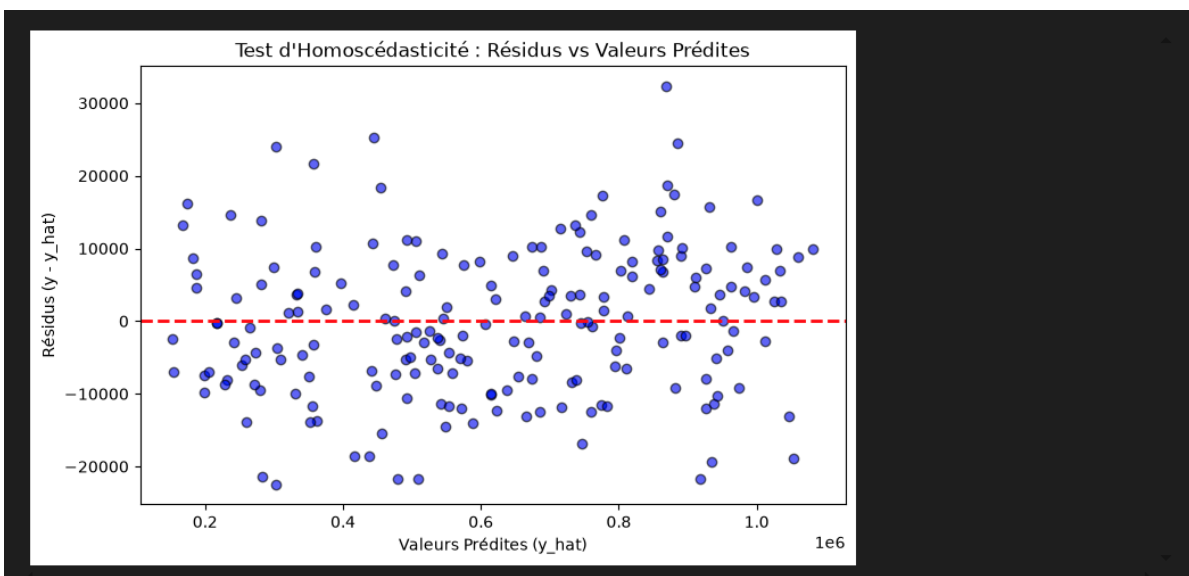
# Prédications et calcul des résidus sur le jeu de TEST
y_pred = modele_sm.predict(X_test_sm)
residus = y_test - y_pred

# Diagnostic 1 : Homoscédasticité (Résidus vs Valeurs prédites)
plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residus, alpha=0.6, color='blue', edgecolors='k')
plt.axhline(0, color='red', linestyle='--', linewidth=2)
plt.title("Test d'Homoscédasticité : Résidus vs Valeurs Prédites")
plt.xlabel("Valeurs Prédites (y_hat)")
plt.ylabel("Résidus (y - y_hat)")
plt.show()

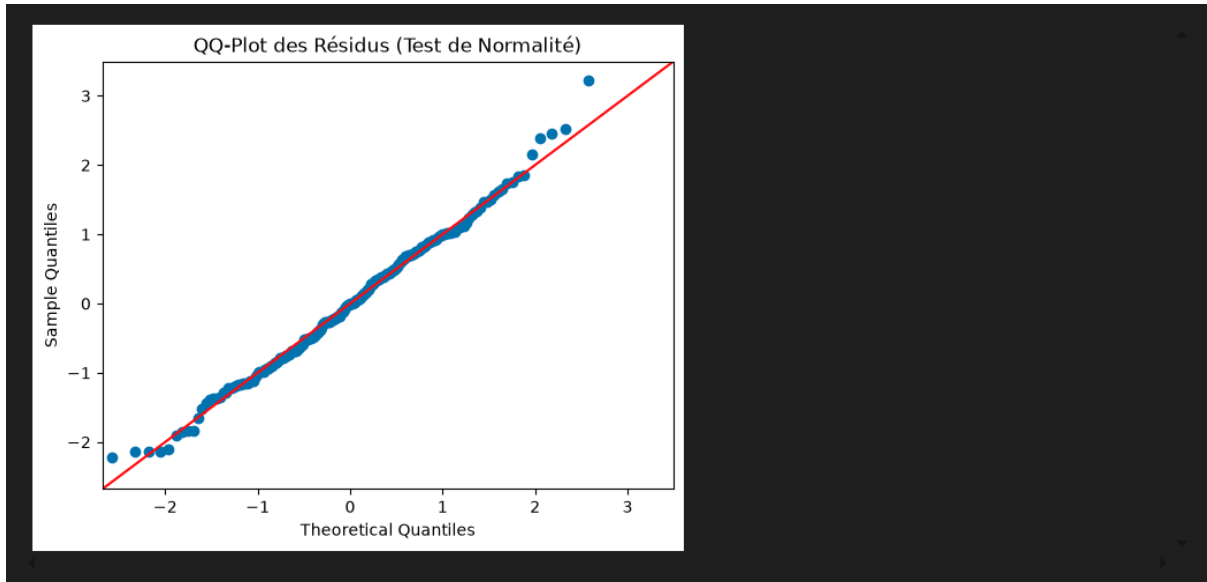
# Diagnostic 2 : Normalité des erreurs (QQ-Plot)
fig = sm.qqplot(residus, line='45', fit=True)
plt.title("QQ-Plot des Résidus (Test de Normalité)")
plt.show()
```

✓ 0.7s Python

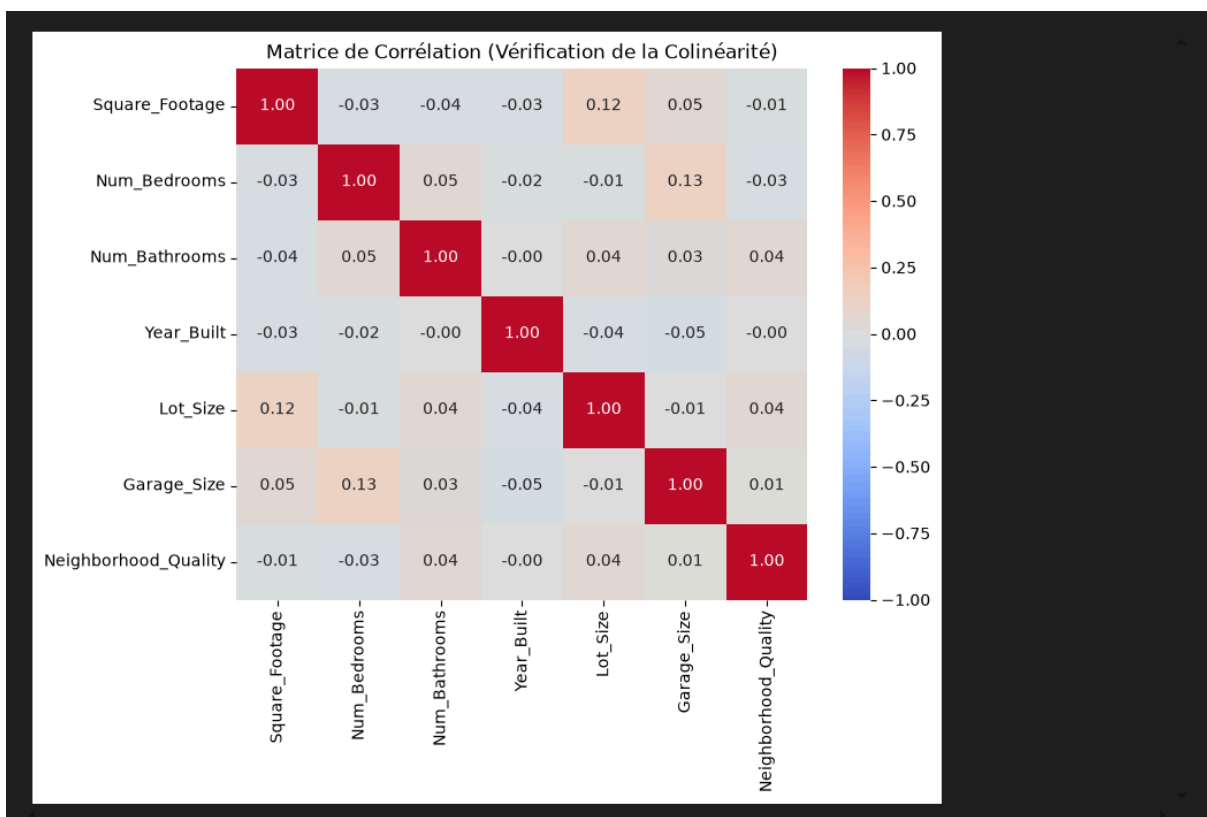
- ☐ Entraîner un modèle de régression linéaire multiple à l'aide de l'ensemble complet des prédicteurs standardisés.
- ☐ Calculer les résidus du modèle sur le jeu de test ($\xi_i = y_i - \hat{y}_i$).
- ☐ Tracer un graphique de dispersion des résidus en fonction des valeurs prédites pour tester l'homoscédasticité. Identifier visuellement s'il y a un motif en entonnoir.



- ☐ Générer un graphique Quantile-Quantile (QQ-Plot) pour valider l'hypothèse de la normalité de la distribution des erreurs.



- ☐ Calculer la matrice de corrélation de vos variables explicatives pour vérifier l'absence de colinéarité excessive.



Étape 4 : Évaluation Statistique et Synthèse

```
print("--- ÉTAPE 4 : Synthèse Statistique du Modèle ---")

# Affichage du rapport statistique complet de statsmodels (R², P-values, Coefficients)
print(modele_sm.summary())

# Vérification manuelle de la généralisation sur le jeu de test
r2_test = r2_score(y_test, y_pred)
n = len(y_test)
p = X_test_scaled.shape[1]
r2_adj_test = 1 - (1 - r2_test) * (n - 1) / (n - p - 1)

print("\n--- Performances sur le jeu de TEST ---")
print(f"R² (Test) : {r2_test:.4f}")
print(f"R² ajusté (Test) : {r2_adj_test:.4f}")
```

- ☐ Calculer et comparer le R^2 et le R^2 ajusté afin d'évaluer la qualité de l'ajustement en pénalisant l'ajout de variables superflues.
- ☐ Analyser les coefficients du modèle pour hiérarchiser l'impact de chaque variable (ex: quel est l'effet d'une hausse de Neighborhood_Quality par rapport à Year_Built ?).
- ☐ Inspecter les p-values associées à chaque coefficient pour identifier les variables statistiquement significatives au seuil de 5 %.
- ☐ Rédiger une conclusion professionnelle résumant les forces et limites du modèle final à destination des décideurs.

```
--- ÉTAPE 4 : Synthèse Statistique du Modèle ---
OLS Regression Results
=====
Dep. Variable:      House_Price      R-squared:      0.999
Model:              OLS              Adj. R-squared: 0.999
Method:             Least Squares    F-statistic:    7.725e+04
Date:               Tue, 11 Aug 2026  Prob (F-statistic): 0.00
Time:               13:22:05          Log-Likelihood: -8478.0
No. Observations:   800              AIC:             1.697e+04
Df Residuals:       792              BIC:             1.701e+04
Df Model:           7
Covariance Type:    nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const          6.186e+05    344.236    1796.956    0.000    6.18e+05    6.19e+05
Square_Footage  2.498e+05    348.002     717.777    0.000    2.49e+05    2.5e+05
Num_Bedrooms    1.452e+04    347.804     41.761    0.000    1.38e+04    1.52e+04
Num_Bathrooms    6695.9068    345.615     19.374    0.000    6017.477    7374.337
Year_Built       2.066e+04    345.141     59.866    0.000         2e+04    2.13e+04
Lot_Size         1.009e+04    347.835     54.877    0.000    1.04e+04    1.98e+04
Garage_Size      4219.4494    348.058     12.123    0.000    3536.223    4902.675
Neighborhood_Quality 335.2475    345.014      0.972    0.331    -342.002    1012.497
=====
Omnibus:          1.958    Durbin-Watson:      2.051
...

--- Performances sur le jeu de TEST ---
R² (Test) : 0.9984
R² ajusté (Test) : 0.9984
```